

Scripting for automation

COMPLETE LEARNING GUIDE

What You'll Learn

There's a wide gap between 'code that produced the right answer once, in a notebook' and 'a script that reliably produces the right answer every day, on new data, unattended.' Closing that gap is what makes a data skill valuable in a real job, and it rests on a specific set of habits: structuring code as a clear extract-transform-load pipeline, exposing configuration through `argparse` instead of hard-coded values, using logging instead of `print()` so failures are diagnosable after the fact, designing for idempotency so a script can be safely re-run without corrupting results, and thinking through how and when the script will actually execute (a schedule, a trigger, retry behavior). This submodule builds a complete small automation script end to end, applying every one of these ideas, so you have a template to adapt for your own data-processing jobs.

1. The Shape of an ETL Script: Extract, Transform, Load

Almost every data automation script — no matter the domain — follows the same three-stage shape. Extract pulls raw data from its source (a file, a database, an API). Transform cleans, validates, and reshapes it into the form the next stage needs. Load writes the result somewhere durable (a database table, a processed-data file, a report). Structuring a script explicitly around these three functions — rather than one long top-to-bottom block — makes it possible to test each stage independently, swap a source or destination without touching the transform logic, and reason about where a failure happened when something goes wrong.

- `extract()` should do as little logic as possible — its only job is 'get the raw data into memory,' so failures there are always about I/O, not business logic.
- `transform()` should be a pure function wherever possible: same input always produces the same output, with no hidden side effects like writing files.
- `load()` is where side effects belong — writing to disk, a database, or an API — kept isolated so the rest of the script stays easily testable.

2. `argparse` in Depth: Turning Constants Into Configuration

Beyond simple `--input/--output` flags, `argparse` supports typed arguments (`type=int`, `type=float`) that fail fast with a clear error if the caller passes something invalid, `choices=` to restrict a flag to a fixed set of valid values, `required=True` for flags that must be supplied, and `default=` for sensible fallbacks. Subcommands (`add_subparsers()`) let a single script expose multiple related operations (e.g. ``pipeline.py clean`` vs ``pipeline.py report``) the way git or docker do, which scales better than a growing pile of boolean flags as a script's responsibilities grow.

PYTHON

```
import argparse

parser = argparse.ArgumentParser(description="Clean and summarize daily sales data")
parser.add_argument("--input", required=True, help="Path to the raw CSV file")
parser.add_argument("--output", default="cleaned.csv", help="Path to write cleaned output")
parser.add_argument("--min-amount", type=float, default=0.0, help="Drop rows less than this amount")
parser.add_argument("--mode", choices=["strict", "lenient"], default="lenient")

args = parser.parse_args()
print(args.input, args.output, args.min_amount, args.mode)
```

3. Logging: Debuggable in Production, Not Just at Your Desk

`print()` statements disappear the moment a terminal closes, have no severity levels, and can't be redirected or filtered. The logging module fixes all three: `logger.info()/warning()/error()` carry a severity level so you can turn verbosity up or down without editing code, a configured handler can write to both the console and a persistent log file simultaneously, and a formatter can automatically stamp every line with a timestamp and the originating module — essential when you're debugging a script that ran unattended at 3am and failed partway through.

PYTHON

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler(), logging.FileHandler("pipeline.log")],
)
logger = logging.getLogger(__name__)

logger.info("Starting pipeline run")
logger.warning("12 rows dropped for missing amount")
logger.error("Failed to write output file: %s", "disk full")
```

4. Idempotency: Designing Scripts That Are Safe to Re-Run

An idempotent script produces the same end state no matter how many times it's run with the same input — critical because scheduled jobs eventually get re-triggered after a partial failure, a retry, or a manual re-run to fix something. Common idempotency techniques include: writing output to a path that includes the run date (so a re-run overwrites that day's file instead of appending duplicate rows), using an upsert (insert-or-update) instead of a blind insert when loading into a database, and checking whether a given input has already been processed (via a manifest file, a processed-flag column, or a content hash) before doing the work again.

- A script that does ``INSERT INTO sales_summary ...`` on every run will silently double-count if it's ever re-run on the same day's data — use UPSERT/MERGE instead.
- Naming output files with the run date (`sales_summary_2026-09-18.csv`) makes re-runs safe by construction — the second run simply overwrites the first.
- A simple manifest file or 'processed_files' table recording what's already been ingested prevents reprocessing the same source file twice.

5. Scheduling Considerations: How the Script Actually Runs

A script that works perfectly when you run it by hand can still fail unpredictably once it's scheduled, because the scheduled environment is different from your interactive shell: environment variables and secrets aren't automatically available, the working directory may not be what you expect (always use absolute paths, never relative ones, in a scheduled script), and there's no human watching to notice a silent failure. On Windows, Task Scheduler runs a script non-interactively; on Linux/macOS, cron does the same — both require the script's exit code to reflect success or failure so the scheduler (or a monitoring layer like Airflow) can detect and alert on problems. Secrets (API keys, database passwords) belong in environment variables or a secrets manager, never hard-coded in the script.

- Always use absolute paths in a scheduled script — a relative path resolves relative to the scheduler's working directory, not the folder the script lives in.
- `sys.exit(1)` on failure and `sys.exit(0)` on success lets a scheduler or orchestrator (cron, Task Scheduler, Airflow) correctly detect and alert on failed runs.
- Environment variables (`os.environ.get('DB_PASSWORD')`) keep secrets out of source code — never commit credentials directly into a script.

6. Error Handling and Retries

A production script should anticipate the specific things that will eventually go wrong — a missing input file, an empty dataset, a network timeout calling an API, a malformed row — and handle each deliberately rather than letting an unhandled exception crash the whole run uninformatively. `try/except` blocks should catch specific exception types (`FileNotFoundError`, `requests.exceptions.Timeout`) rather than a bare `except:`, which can silently swallow bugs unrelated to the problem you intended to handle. For transient failures (a flaky network call), a small retry loop with a short delay is often enough; for anything else, fail loudly and log enough context to diagnose the problem later rather than continuing silently with bad data.

- Catch specific exceptions (`FileNotFoundError`, `ValueError`) rather than a bare `except:` — a bare `except` hides bugs you didn't intend to handle.
- For network calls, a small `retry-with-backoff` loop (try up to 3 times, waiting longer each time) handles the common case of a transient failure without masking a persistent one.
- When a row of data is malformed, log it and skip it rather than crashing the entire run — but always log a count so silent data loss doesn't go unnoticed.

7. Full Worked Example: A Small End-to-End Automation Script

Putting every piece together — argparse for configuration, logging for observability, idempotent output naming, and structured error handling — produces a script that's genuinely safe to schedule and forget about, rather than one that only works when you're watching it run.

PYTHON

```
import argparse
import logging
import sys
from datetime import date
from pathlib import Path

import pandas as pd

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s]
%(message)s")
logger = logging.getLogger("daily_sales_pipeline")

def extract(input_path: Path) -> pd.DataFrame:
    if not input_path.exists():
        raise FileNotFoundError(f"Input file not found: {input_path}")
    return pd.read_csv(input_path, dtype={"customer_id": str})

def transform(df: pd.DataFrame, min_amount: float) -> pd.DataFrame:
    before = len(df)
    df = df.dropna(subset=["amount"])
    df = df[df["amount"] >= min_amount]
    logger.info("Dropped %d invalid/low-value rows", before - len(df))
    return df.groupby("customer_id", as_index=False)["amount"].sum()

def load(df: pd.DataFrame, output_dir: Path) -> Path:
    output_dir.mkdir(parents=True, exist_ok=True)
    out_path = output_dir / f"sales_summary_{date.today().isoformat()}.csv"
    df.to_csv(out_path, index=False) # date-stamped path -> idempotent re-runs
    return out_path

def main() -> int:
    parser = argparse.ArgumentParser(description="Summarize daily sales by customer")
    parser.add_argument("--input", required=True, type=Path)
    parser.add_argument("--output-dir", default=Path("./output"), type=Path)
    parser.add_argument("--min-amount", type=float, default=0.0)
    args = parser.parse_args()

    try:
        raw = extract(args.input)
        summary = transform(raw, args.min_amount)
        out_path = load(summary, args.output_dir)
        logger.info("Wrote summary for %d customers to %s", len(summary), out_path)
        return 0
    except Exception:
        logger.exception("Pipeline run failed")
```

```
    return 1
```

```
if __name__ == "__main__":  
    sys.exit(main())
```

Common Pitfalls

- Hard-coding file paths, thresholds, or credentials directly in the script instead of exposing them as arguments or environment variables.
- Using `print()` for status output, leaving nothing to inspect after an unattended scheduled run fails overnight.
- Writing a script that isn't idempotent — a re-run after a partial failure double-processes or double-inserts data.
- Catching bare `Exception` (or worse, a bare `except:`) and continuing silently, hiding real bugs.
- Using relative file paths, which resolve differently depending on the scheduler's working directory versus your interactive shell.
- Not setting a non-zero exit code on failure, so a scheduler or orchestrator has no way to detect and alert on a failed run.
- Forgetting to handle the empty-input or missing-file case, causing a cryptic downstream error instead of a clear, early failure message.

Key Takeaways

- Structure automation scripts as extract/transform/load functions — it makes each stage independently testable and swappable.
- `argparse` turns 'edit the code and re-run' into 'pass a flag' — a small change that makes a script genuinely reusable by others.
- logging, not `print()`, is what makes an unattended scheduled failure diagnosable after the fact.
- Idempotent design (date-stamped outputs, upserts, processed-file manifests) is what makes a script safe to schedule and forget.
- A scheduled script needs absolute paths, environment-variable secrets, and a correct exit code — none of which matter when you run it by hand.
- Handle the specific failures you can anticipate (missing file, empty dataset, flaky network call) rather than letting one unhandled exception crash the whole run.

Learn More

Python Docs — argparse Tutorial

<https://docs.python.org/3/howto/argparse.html>

Python Docs — logging HOWTO

<https://docs.python.org/3/howto/logging.html>

Real Python — Automating Scripts With argparse and logging

<https://realpython.com/command-line-interfaces-python-argparse/>